



TaskFlow

Architecture Document

Clean Flutter architecture • BLoC state management • local mock data • secure simulated authentication

Version 1.0

Android-first

23 August 2026

Document Contents

01 Purpose and scope	09 Authorization and security
02 Architecture overview	10 Navigation
03 Folder structure	11 Responsive design system
04 State management	12 Testing strategy
05 Mock data and repositories	13 Build readiness
06 Authentication flow	14 Decisions and trade-offs
07 Local storage	15 Known limitations
08 Error and offline handling	16 Conclusion

Version: 1.0

Platform: Flutter, Android-first

State management: BLoC / Cubit

Data source: Bundled local mock JSON

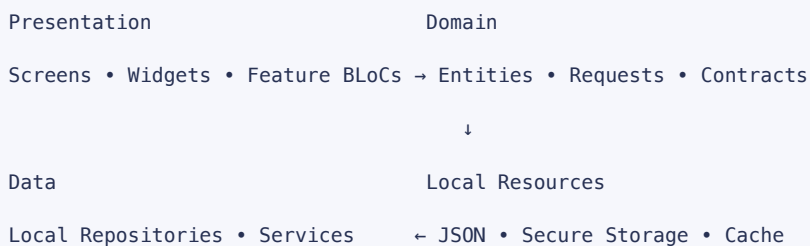
1. Purpose and scope

TaskFlow is an Android-first project and task management application built from the supplied mock data. It provides simulated JWT-style authentication, organization-scoped projects, task management, member assignment, notifications, responsive layouts, offline awareness, local caching, error simulation, biometric unlock, and inactivity timeout—without calling a live backend.

This document describes the implemented architecture, dependency boundaries, state management, data and authentication flows, local storage, navigation, error handling, security rules, testing strategy, and important engineering decisions.

2. Architecture overview

TaskFlow uses a clean, layered architecture with inward-facing contracts and constructor-based dependency injection.



The UI never reads JSON or storage directly. Widgets dispatch BLoC events; BLoCs call domain repository contracts; local repository implementations enforce validation and authorization before accessing the data source or storage. A future HTTP implementation can replace the local repositories without changing screens.



3. Folder structure and responsibilities

```
lib/
├─ core/                brand, responsive, validation, localization,
                        |
                        | cancellation and activity monitoring
├─ data/
  │ └─ datasources/     centralized mock JSON boundary
  │ └─ repositories/    local repository implementations
  │ └─ services/        platform biometric adapter
├─ domain/
  │ └─ models/          entities, requests, responses and filters
  │ └─ repositories/    backend-independent contracts
  │ └─ services/        platform-independent contracts
├─ presentation/
  │ └─ blocs/           auth, projects, tasks, notifications, settings
  │ └─ screens/         feature screens and forms
  │ └─ widgets/         reusable UI components
└─ main.dart            composition root and dependency wiring
```

`main.dart` constructs the data source, storage-backed repositories, and platform services, then provides feature BLoCs with `MultiBlocProvider`. Explicit constructor injection keeps dependencies visible and replaceable in tests.

4. State management

The app uses `flutter_bloc` with bounded feature ownership.

Component	Responsibility
<code>AuthBloc</code>	Session check, login, registration, biometric gating, refresh, logout, inactivity timeout
<code>ProjectsBloc</code>	Organization project reads, create/edit/delete, refresh, errors
<code>TasksBloc</code>	Tasks, filters, CRUD, status/priority, assignment, members
<code>NotificationsBloc</code>	Inbox loading, unread state, read mutations
<code>SettingsCubit</code>	Theme, locale, simulated connectivity/error controls

Feature states use `LoadPhase` consistently:

```
Initial → Loading → Success
    ↘
    ↘ Empty
    ↘ Error → Retry → Loading
```

Mutation logic, validation, and access control stay outside widgets. `setState` is limited to ephemeral UI details. Authentication changes coordinate organization-scoped feature loads and clear protected feature state on logout.

5. Mock data and repository layer

The complete fixture is bundled as one Flutter asset: `assets/mock-data.json`. `AssetMockDataSource` is the only component that loads it. Each top-level key is parsed into its own typed collection: organizations, users, organization members, projects, tasks, comments, notifications, credentials, and token response.

```
Screen → BLoC event → Repository interface → Local repository
                                     |
                                     |— validation
                                     |
                                     |— authorization
                                     |
                                     |— organization scope
                                     |
                                     |— AssetMockDataSource
                                     |
                                     |— local cache / secure session
```

The data source centralizes writes through project/task upsert and removal, task assignment, and membership mutation methods. Repositories expose request/response-style objects and typed models with JSON serialization. Every project, task, and member read is scoped to the authenticated user's `org_id`.

Mutations update process-local state as permitted by the assignment. Successful project/task responses are cached separately for offline display.

6. Simulated authentication flow

```
App launch → Splash / session check
├─ no secure session → Login
└─ stored session
    ├─ token valid → optional biometric gate → Home
    └─ token expired → mock refresh → Home
        └─ refresh unavailable → Login
```

Login credentials and the initial mock token response are loaded from `auth_mock` in the bundled JSON—not hardcoded in UI widgets. On success, the access token, refresh token, expiry, and session metadata are stored with `flutter_secure_storage`. Passwords are never persisted and tokens are never logged.

Access expiry is derived from `access_token_expires_in_seconds` (900 seconds / 15 minutes). An expired restored session performs a simulated refresh that creates a distinct JWT-style access token and renews expiry while retaining the fixture refresh token.

Bonus protections are implemented:

- Biometric unlock can gate restoration of an existing session.
- Biometric cancellation offers retry and password-login fallback.
- Five minutes of inactivity triggers automatic logout.
- Touch, pointer, and keyboard activity reset the timer; resume checks background elapsed time.
- Logout deletes secure state, clears feature BLoCs, and blocks back navigation to authenticated screens.

7. Local storage

Storage	Purpose	Classification
Flutter Secure Storage	Access/refresh tokens, expiry, session metadata, biometric preference	Sensitive
SharedPreferences	Last successful project/task snapshots and UI/mock preferences	Non-sensitive
Process memory	Parsed fixture and current local mutations	Non-sensitive

No password, production secret, signing key, or live service credential is stored. Cached snapshots support stale reads; they are not a transactional database.

8. Error handling, delay, offline, and cancellation

The local data layer deliberately behaves like a remote boundary:

Condition	Trigger	Behavior
Artificial latency	Every mock read, randomized 300–800 ms	Loading indicators and skeletons
Timeout	Settings → Simulate timeout	Typed error and retry
Offline	Settings → Simulate offline	Preserved/cached data, stale warning, retry
404	Unknown project/task ID	Typed not-found failure
Validation	Empty or invalid request	Field feedback plus repository rejection
Authorization	Non-admin or cross-org mutation	Blocked below the UI
Cancellation	New refresh supersedes active read	Old request stops without a false error

When offline, already-loaded data remains visible; cached data is marked potentially stale. Without a cache, an offline error is shown instead of a crash. Reconnection refreshes authoritative state. Offline mutation queuing is intentionally not included.

9. Authorization and security

Only `org_admin` may delete projects or manage members. The UI communicates eligibility, but the repository independently blocks crafted BLoC events or direct-navigation attempts. Self-removal is blocked; removing a member unassigns affected tasks. Assignment validates organization membership even though the picker already filters choices.

Security boundaries:

- No real/live backend, third-party API, Firebase, or Backend-as-a-Service.
- No password storage or token logging.
- No credential logic hardcoded in widgets.
- No embedded production secrets or release signing keys.
- Protected screens are replaced after logout, expiry, or inactivity timeout.
- Biometrics are hidden behind a testable domain service interface.

10. Navigation

```
Splash → Login ↔ Register
      |
      | authenticated
      ▼
Home shell
├─ Dashboard — upcoming task —————> Task Details
├─ Projects — Project Details —————> Task Details / Project Form
├─ Tasks — Task Details —————> Create/Edit Task
└─ Settings — Profile / Members / Notifications / Security
```

The root `MaterialApp` selects Splash, Login, or Home from `AuthBloc`. The Home shell uses bottom navigation on compact devices and a navigation rail on larger layouts. Notifications deep-link to tasks, destructive actions require confirmation, and logout clears the protected navigation tree.

11. Responsive UI and design system

The shared TaskFlow palette in `app_colors.dart` drives buttons, navigation, accents, and branded surfaces using tints and opacity rather than unrelated colors. Responsive helpers provide compact, medium, and expanded breakpoints, centered maximum widths, adaptive project grids, flexible dashboard metrics, scroll-safe forms, and landscape-safe filters.

Bonus polish includes dark mode, tablet layout, custom animations, semantic skeleton loading, screen-reader labels, large-text-safe layouts, English/Hindi localization, and bundled user avatars that avoid a runtime third-party image dependency.

12. Testing strategy

Tests are isolated, order-independent, and use fresh mock-backed dependencies; they never require a real network.

Level	Coverage
Unit	Auth/session refresh, biometrics, inactivity, validation, filters, BLoCs, authorization, cancellation, serialization, fixture parsing, notifications
Widget	Login validation, task loading/empty/error/success, status update, project refresh, confirmations, responsive layout, localization, accessibility, golden login
Integration	Mock login, organization projects, task listing, create/update task, assignment

The current suite passes **41 tests**, and `flutter analyze` reports **no issues**. Coverage is generated with `flutter test --coverage` into `coverage/lcov.info`.

13. Build and production readiness

Verified toolchain: Flutter 3.41.9 stable, Dart 3.11.5, DevTools 2.54.2.

```
flutter pub get
flutter run
flutter analyze
flutter test
flutter test integration_test/app_test.dart
flutter test --coverage
flutter build apk --release
```

The app starts from bundled assets and local storage without source modifications, environment files, or live-service configuration. Android debug and release paths have been verified. The reviewer APK is generated at `build/app/outputs/flutter-apk/app-release.apk`.

Google Play publication requires replacing development signing with the owner's private upload/release key stored outside source control.

14. Key decisions and trade-offs

Decision	Reason	Trade-off
Feature BLoCs	Predictable, testable state	More ceremony than widget-local state
Repository interfaces	Future HTTP replacement without UI changes	Added abstraction for a local app
Single bundled fixture	Deterministic and assignment-compliant	Original source remains immutable
Process-local writes	Matches no-backend scope	CRUD resets with process data
SharedPreferences snapshots	Simple stale/offline reads	Not a transactional database
Secure token storage	Models safe session handling	Tokens are simulated and unsigned
Repository authorization	Prevents UI-only bypass	Intentional duplicate eligibility checks
Cooperative cancellation	Avoids stale completion/noisy errors	Simulates cancellation rather than sockets

15. Known limitations

- Android is the required and verified target; iOS is optional and not treated as a release target.
- JWT-style tokens and refresh are simulated; no cryptographic signing/server validation occurs.
- Registration succeeds locally but does not create a durable credential.
- Project/task/member mutations are process-local; cache preserves reads, not transactional writes.
- Offline-first mutation queuing and later synchronization are not implemented.
- Assignment changes do not synthesize new notification fixture events.
- Fixture text remains in its original language when the app chrome is Hindi.
- Biometric behavior depends on device support and enrollment.

16. Conclusion

TaskFlow separates presentation, business logic, and data access; uses explicit dependency injection; centralizes the mock asset behind repository contracts; secures simulated session data; enforces organization authorization below the UI; and provides tested loading, empty, success, error, stale, and cancellation behavior. The structure is ready for a future HTTP repository while remaining deterministic and offline for assignment review.